Introduction

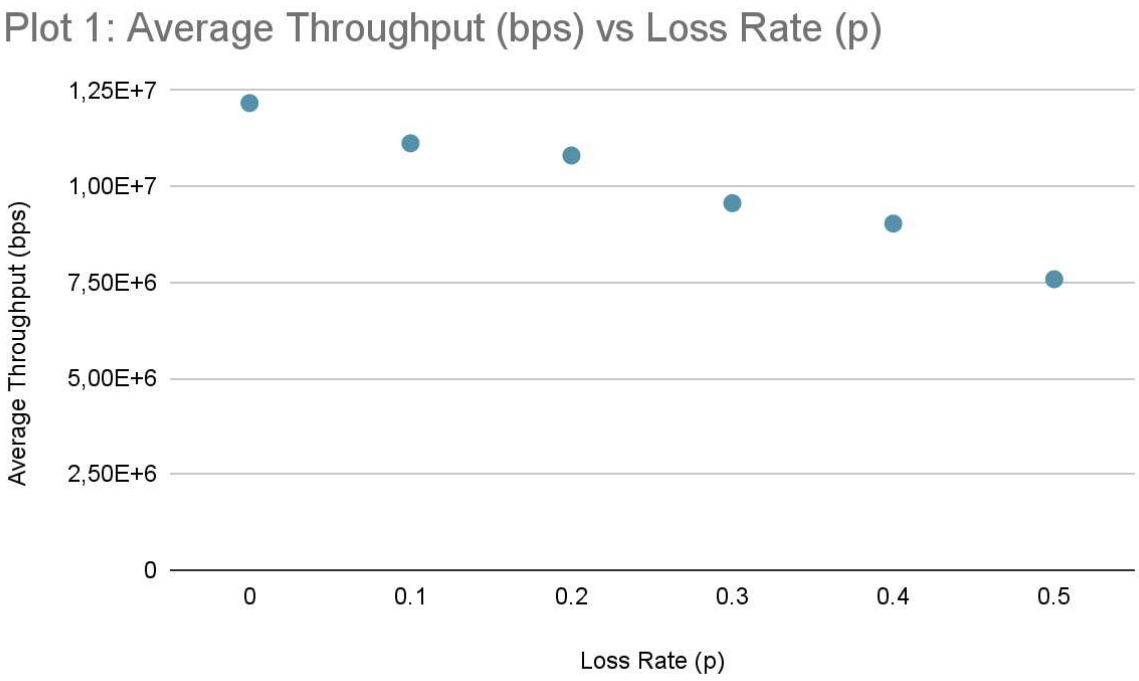
In this programming assignment, we have implemented a sender using GBN via UDP to transfer an image to the receiver. The throughput of the sending process varies when different loss rates, timeouts, or window sizes are used. To demonstrate the differences and see their impacts on the throughput, we will evaluate the three plots below and discuss their meanings.

To calculate the throughputs, we will use the following formula:

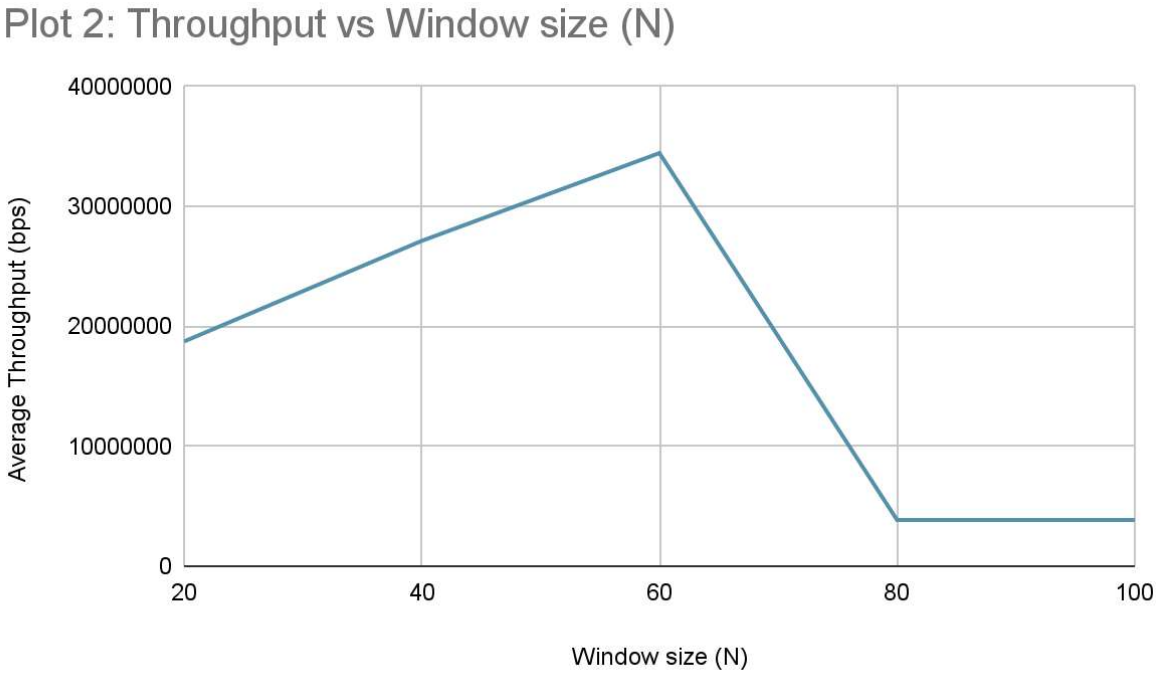
$$throughput\ (bps)\ =\ file_size_bits\ /\ total_transfer_time_seconds.$$

To accurately calculate the total transfer time, each run is repeated three times, and the average of these three runs is used as the denominator. The size of the image is 34,134,832 bits (4,266,854 bytes). We can use this two information to calculate the throughput and plot our graphs.

Plots



As you can see, plot 1 shows a decrease in throughput when the loss rate increases. This is expected when GBN is used. This is because with GBN, the sender can have multiple packets in flight, but the receiver only accepts them in order. When a packet or an ACK is lost, packets sent later will be dropped by the receiver. When the timeout occurs, the sender will resend the packet after the latest received packet. As the loss rate grows, the frequency of losses increases, and this causes an increase in the number of retransmissions, the number of duplicate packets sent, and the amount of time spent recovering. Therefore, throughput decreases when the loss rate increases.

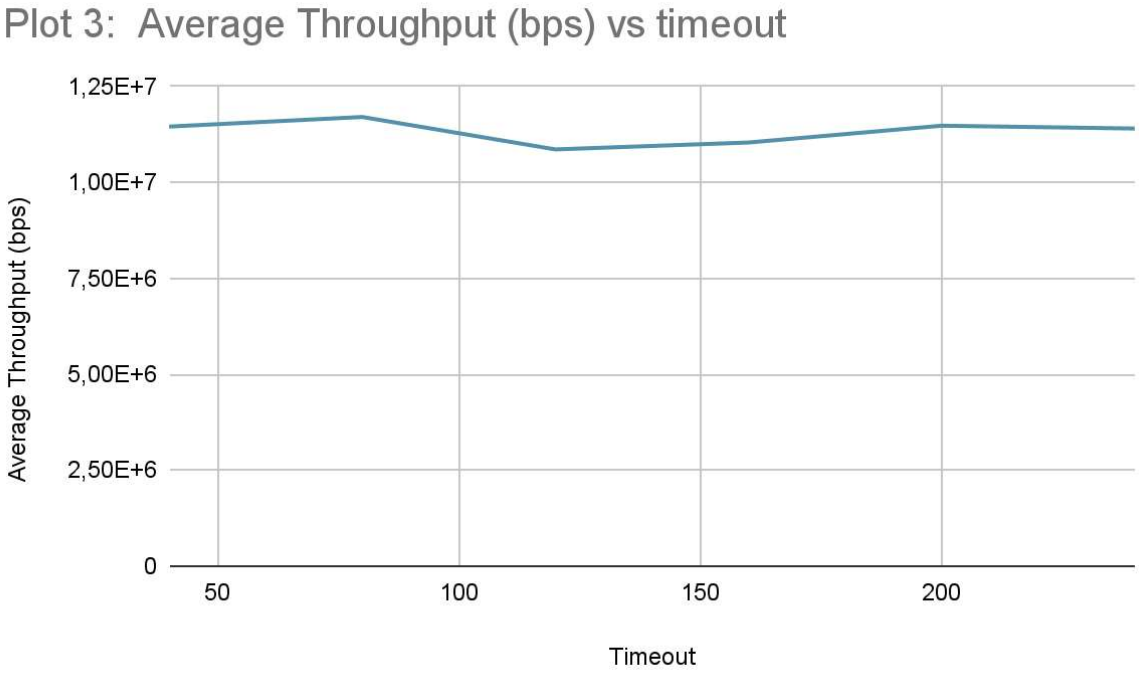


In plot 2, we see that the throughput increases from N=20 up to N=60, then drops for N=80 and N=100. This behavior is also consistent with GBN and what we learn about the trade-off between pipelining and retransmission cost.

- For smaller windows, increasing N improves throughput because the sender can pipeline more packets without waiting for each ACK. This reduces idle time and better uses the available link capacity.

- However, in GBN, a larger window makes each loss more expensive since one lost packet can force the retransmission of many outstanding packets in the window. Delayed ACKs and non-zero loss, along with a large window, increase the chance that the sender has a lot of unacknowledged packets when a loss occurs, which leads to larger retransmission bursts.

The big drop at N=80 and N=100 suggests that beyond a certain window size, the sender spends a lot more time retransmitting, which reduces throughput. This shows that larger windows are not always better in protocols like GBN under loss; there is often an optimal window size that balances the benefits of pipelining against the overhead of retransmission.



In plot 3, we see that the throughput stays high and changes only slightly across different timeout values. This is reasonable given the experiment setup. Timeout controls how quickly the sender decides that a packet or ACK is lost and then retransmits:

- If the timeout is too small compared to the RTT, the sender retransmits packets that were not lost in the first place, increasing duplicate traffic and lowering throughput.

- If the timeout is too large, then when a real loss occurs, the sender waits longer before retransmitting, which can slow recovery and lower throughput.

In our results, throughput is relatively stable, so this could mean that across 40–240 ms, the timeout values were generally not too small or too large for the observed network delay behavior. Small differences across the curve can be explained by randomness from simulated loss/delay and occasional timeouts/retransmissions. Overall, this matches the idea that a good timeout should be on the same order as the RTT/delay.

Conclusions

In conclusion, the plots demonstrate the core GBN trade-offs: throughput decreases with loss due to retransmissions (Plot 1), window size improves pipelining only up to a point before retransmission costs dominate (Plot 2), and timeout needs to be chosen relative to delay/RTT to avoid unnecessary retransmissions or slow loss recovery (Plot 3).